



Week 1 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@September 4, 2021 7:51 PM
🔗 Materials	
# Module #	1
▼ Type	Lecture
☰ Week #	1

What is an App?

"An app is a computer software, or a program, most commonly a small, specific one used for mobile devices. The term app originally referred to any mobile or desktop application, but as more app stores have emerged to sell mobile apps to smartphone and tablet users, the term has evolved to refer to small programs that can be downloaded and installed at once"

- Source: Techopedia

Desktop Apps

- Usually standalone
 - Editors / Word processors
 - Web browsers
 - Mail clients
- Often work offline
 - Local data storage
 - Possible network connection
- Software Development Kits (SDK)
 - Custom frameworks
 - OS specific

Mobile Apps

- Targeted at mobile platforms: Phones / Tablets

- Constraints
 - Limited screen space
 - User interaction (touch, audio, camera)
 - Memory / processing
 - Power
- Frameworks
 - OS specific
 - Cross-platform
- Network!
 - Usually network oriented

Web Apps

- The **Platform**
- Works across OS, device: create a common base
- Heavily network dependent
 - Workarounds for offline processing

Our main focus in this course ...



Week 1 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@September 4, 2021 8:10 PM
🔗 Materials	
# Module #	2
▼ Type	Lecture
☰ Week #	1

Components of an App

Storage

Computation

Presentation

Example

- Storage
 - Where are the e-mails stored?
 - How are they stored on the server? File formats, etc.
- Compute
 - Indexing of e-mails
 - Searching
- Presentation
 - Display list of e-mails
 - Rendering / display of individual e-mails

Platforms

- Desktop
 - Keyboard, Mouse, Video as I / O
 - Desktop paradigm - folders, files, documents

- Mobile
 - Touchscreen as I / O
 - Voice, tilt, camera interfaces
 - Small self-contained apps
- Web-based
 - Datacenter storage - persistent
 - Cloud: access anywhere multi-device
- Embedded
 - Single-function, limited scope



Week 1 Lecture 3

▼ Class	BSCCS2003
🕒 Created	@September 5, 2021 7:38 PM
🔗 Materials	
# Module #	3
▼ Type	Lecture
☰ Week #	1

Client-Server & Peer-to-Peer Architecture

Architectures

- Client-Server
 - Server:
 - Stores data
 - Provides data on-demand
 - May also perform computations
 - Client:
 - End users
 - Requests data
 - User interaction, display
 - Network:
 - Connects the client to the server
 - Can be local
 - Data pipe - no alterations
 - In a client-server model, the server and client are explicitly defined
 - Local systems:
 - Both client and the server on the system ⇒ local network / communication

- Conceptually, it is still a networked system
- Machine clients
 - eg: Software, antivirus updaters
 - They need not have user interaction
- Variants
 - Multiple servers, single queue, multiple queues, load balancing front-ends
- Examples:
 - E-mail
 - Databases
 - WhatsApp / messaging
 - Web browsing

- **Distributed - Peer-to-Peer**

Data can flow both ways

- All peers are considered "equivalent"
 - But some peers may be more equal than others

This just means that some servers with high bandwidth are given higher weightage
- Error tolerance
 - Masters / Introducers
 - Election / re-selection of masters on failure
- Shared information
- Examples:
 - BitTorrent
 - Blockchain-based systems
 - IPFS, Tahoe (distributed file system).



Week 1 Lecture 4

▼ Class	BSCCS2003
🕒 Created	@September 5, 2021 8:33 PM
🔗 Materials	
# Module #	4
▼ Type	Lecture
☰ Week #	1

Software Architectures

What is a design pattern?

"A general, re-usable solution to a commonly occuring problem within a given context in software design"

Source: Wikipedia

- Experienced designers observe "patterns" in code
- Re-using these patterns can make design and development faster
- Guide the design and thought process

M-V-C paradigm

- **Model:**
 - Core data to be stored for the application
 - Databases: indexing for easy searching, manipulation, etc;
- **View:**
 - User-facing side of the application
 - Interfaces for finding information, manipulating
- **Controller:**
 - "Business Logic" - how to manipulate the data

Origins in the smalltalk language, 1979

The user uses the **controller** → to manipulate the **model** → that updates the **view** → that the user sees;

Other design patterns

- Model-View-Adapter
- Model-View-Presenter
- Model-View-ViewModel
- Hierarchical MVC
- Presentation-Abstraction-Control

Each one of them has its own uses, but the fundamentals are very similar

Focus of this course

- Platform:
 - Web-based
- Architecture:
 - Client-server
- Software architecture
 - Model-View-Controller



Week 1 Lecture 5

▼ Class	BSCCS2003
🕒 Created	@September 6, 2021 12:07 PM
🔗 Materials	
# Module #	5
▼ Type	Lecture
☰ Week #	1

Introduction to the Web

- Platform of choice for this course
- Generic - works across operating systems, hardware architectures
 - Can it be considered as a cross-platform OS?
- Built on sound underlying principles
- Worth understanding
 - Constraints: what can and cannot be done (easily)
 - Costs: storage, network, device sizing, datacenter

Historical background

- Telephone networks ~ 1890+
 - Circuit switched - allow A to talk to B, through complex switching network
 - Physical wires tied up for duration of call even if nothing was said!
 - The bandwidth used by voice signal is pretty low
- Packet switched networks ~ 1960s
 - Wires occupied only when data was to be sent - more efficient use

Last link: The physical connection between my telephone and my nearest telephone exchange, it has to be tied up to me

Aggregate all the data in one place (telephone exchange) and transmit it over a single wire at a much higher speed (connection between telephone exchanges)

- Data is being sent instead of voice over the wires
- ARPANet - 1969
 - acronym for Advanced Research Projects Agency Network
 - Node-to-node network
 - Mostly university driven
- Others:
 - IBM SNA, Digital DECNet, Xerox Ethernet, ...
- Protocol
 - How to format packets; place them on a wire; headers / checksums;
 - Each network had its own protocol
- "Inter" network
 - How to communicate between different network protocols?
 - Or replace them with a single "inter"net protocol
- IP: Internet Protocol - 1983
 - Define headers, packet types, interpretation
 - Can be carried over different underlying networks: ethernet, DECnet, PPP, SLIP
- TCP: Transmission Control Protocol - 1983
 - Establish reliable communication - retry; error control
 - Automatically scale and adjust to network limits
 - It is a connection oriented protocol
- Domain names ~ 1985
 - Use names instead of IP addresses
 - IP addresses were basically a set of 4 numbers, what they called the dotted quad notation
 - There are 4 numbers in the range of 0 - 255, which are separated by dots (.)
 - Easy to remember - *.com* revolution still in the future
- HyperText ~ 1989+
 - Text documents to be "served"
 - Formatting hints inside documents to "link" to other documents - HyperText
 - It was brought into one consolidated framework by Tim Berners Lee at CERN
 - All this lead to the creation of the World Wide Web

Where are we now?

- Original web was limited to ...
 - static pages
 - complicated executable interfaces
 - limited styling
 - browser compatibility issues
- Web 2.0 ~ 2004+
 - dynamic pages - generated on the fly
 - HTTP as a transport mechanism - binary data; serialized objects
 - client side computation and rendering
 - platform agnostic operating system



Week 1 Lecture 6

▼ Class	BSCCS2003
🕒 Created	@September 6, 2021 2:01 PM
🔗 Materials	
# Module #	6
▼ Type	Lecture
☰ Week #	1

How does the Web work?

Digging deeper

- What is a server?
- What is a transport protocol?
- Impact on performance:
 - Network
 - Server compute resources
 - Storage requirements
 - Client compute resources

Web Server

- Any old computer with a network connection can act as a web server
- Software:
 - Listen for incoming network connections on a fixed port
 - Respond in specific ways
 - Opening network connections, ports etc already known to OS
- Protocol:
 - What should client ask server
 - How should the server respond to the client

HTTP

- HyperText
 - Regular text documents
 - Contains "codes" inside that indicate special functions - how to "link" to other documents
- HyperText Transfer Protocol
 - Largely text based: client sends requests, server responds with hypertext documents



Week 1 Lecture 7

▼ Class	BSCCS2003
🕒 Created	@September 6, 2021 2:43 PM
🔗 Materials	
# Module #	7
▼ Type	Lecture
☰ Week #	1

Simple Web Server

Simplest server

```
#!/bin/bash

while true; do
  echo -e "HTTP/1.1 200 OK\n\n $(date)" |
  nc -l localhost 1500;
done
```

Create a new file on the linux terminal

```
$ nano server.sh
```

Write the above mentioned server bash script in that file, and then to run the script, type the following code on the linux terminal

```
$ bash server.sh
```

We can send a request to the running server from another instance of linux shell

```
$ curl http://localhost:1500
```

We can also add a verbose flag to get a ton more info


```
$ curl -v http://localhost:1500
```

What the general web server is actually doing?

- Listening on a fixed port
- On incoming requests, it ran some code, generated results on the fly and returned the results to the client
 - Standard headers to be sent as a part of the result
 - Output can be in text or other format - MIME (Multipurpose Internet Mail Extensions)

What does a typical request look like?

```
GET / HTTP/1.1
Host: localhost:1500
User-Agent: curl/7.64.1
Accept: */*
```

Viewing responses with curl

- curl, wget, etc - simple command line utilities
- Can perform full HTTP requests
- Verbose output includes all the headers
- Very useful for debugging!

What does a typical response look like?

When you make a simple http request to a web server via curl, the following response appears ...

```
*   Trying ::1...
* TCP_NODELAY set
* Connected to localhost (::1) port 1500 (#0)
> GET / HTTP/1.1
> Host: localhost:1500
> User-Agent: curl/7.64.1
> Accept: */*
>
< HTTP/1.1 200 OK
* no chunk, no close, no size. Assume close to signal end
<
Thu Jun 17 08:14:55 IST 2021
```

NOTE: The `:: 1` is an IPv6 address which represents *localhost*



Week 1 Lecture 8

▼ Class	BSCCS2003
🕒 Created	@September 6, 2021 3:26 PM
🔗 Materials	
# Module #	8
▼ Type	Lecture
☰ Week #	1

What is a Protocol?

- When both sides agree on how to talk
- Server expects "requests"
 - Nature of requests
 - Nature of client
 - Types of results that the client can deal with
- Client asks "responses"
 - Asks server for something
 - Convey what you can accept
 - Read the result and process

HyperText Transfer Protocol (HTTP)

- Primarily text based
- Requests specified as "GET", "POST", "PUT", etc
 - Headers can be used to convey acceptable response types, languages, encoding
 - Which host to connect to (if multiple hosts are there on a single server)
- Response headers
 - convey message type, data
 - cache information

- status codes: 404 not found, etc

Use cases

- **GET**: simple requests, search queries, etc
- **POST**: more complex form data, large text blocks, file uploads
- **PUT / DELETE / ...**
 - Rarely used in Web 1.0
 - Extensively used in Web 2.0
 - Basis of most APIs - REST, CRUD

Another web server

```
$ python -m http.server
```

- Serve files from local directory
- Understands basic HTTP requests
- Gives more detailed headers and responses

When the above command is run in a given directory, if I find a file called *index.html*, return me that file else return me the directory listing



Week 1 Lecture 9

▼ Class	BSCCS2003
🕒 Created	@September 6, 2021 3:44 PM
🔗 Materials	
# Module #	9
▼ Type	Lecture
☰ Week #	1

Performance of a Website

- How fast a site can be?
- What limits the performance?
- Some basic observations ...

Latency

How much time does it take to get the response for a given request

- Speed of light: $3 \times 10^8 m/s$ in vacuum
- $\sim 2 \times 10^8 m/s$ on a cable
 - $\sim 5ns$ for 1m $\implies \sim 5ms$ for 1000km
- Datacenter is 2000km away
 - One-way request: $\sim 10ms$
 - Round-trip: 20ms

So, what does this mean?

If I am continuously sending requests to a server, and I want it such that whenever I send a request, I get a response back before I can then proceed, then

I am limited to a maximum of 50 requests per second

What about a server that needs to send back responses?

Response size

- Response: 1KB of text (headers, HTML, CSS, JS)
- Network connection = 100Mbps (Megabits per second) $\approx$ 10MBps (Megabytes per second) [Theoretically, it should be 12.5MB/s]
- This limits us to $\sim$ 10,000 requests per second

For eg: Just imagine, that the result of some entrance exams were declared in India and if it is all trying to come out of one server with the bandwidth of 100 Megabits per second

The server would simply crash when it is bombarded with more than 10,000 requests per second

A simple case study: Google response

- Headers: $\sim$ 100B (bytes)
- Content: 144kB (kilobytes)
- Approx. 60,000 requests per second

Which means that Google server effectively need more than 80Gbps of bandwidth

Memory: YouTube

- One python HTTP server process: $\sim$ 6MB (measured)
- Multiple parallel requests: multiple processes
 - eg: YouTube will have long running server processes
- 2016 Presidential debate in the US:
 - 2M+ concurrent viewers

Storage: Google

- Index 100s of billions of web pages
- Cross-reference, [pagerank](#)
- Total index size estimate:
 - 100,000,000 Gigabytes $\implies$ 100 Petabytes
- Storage?
- Retrieval

Summary

- The Web is a useful device / OS agnostic platform for apps
- Built for HTTP for transport, HTML and related tech for presentation
- Servers trivial at most basic
- Scaling requires careful design



Week 2 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@September 14, 2021 12:16 PM
🔗 Materials	
# Module #	10
▼ Type	Lecture
☰ Week #	2

Information Representation in a Machine

Markup

- Information representation
- Raw data vs Semantics
- Logical structure vs Styling
- HTML5 & CSS

Information representation

- Computers work only with "bits"
 - Binary digits: 0 and 1
- Numbers
 - Place value: binary numbers: eg. 6 = 0110
 - Two's complement: negative numbers: eg. -6 = 1010

Representing Text

- ASCII
- Unicode
- UTF-8

Information Interchange

- Communicate through machines - either between machines or between humans
- Machines only work with **bits**
- Standard "encoding"
 - Some sequence of bits interpreted as a character

Interpretation

What is "0100 0001"?

- It is a string of bits
- Number with a value of 65 in decimal
- Character "A"
- All of the above statements are correct

It boils down to a matter of **interpretation** and **context**

ASCII

American Standard Code for Information Interchange

- 7-bits: 128 different entities
 - 'a'..'z'
 - 'A'..'Z'
 - '0'..'9'
 - Special characters- ! @ # \$ % ^ & * () ...
- Why 7-bits?

It was a time before even 8-bit computers were standardised

The number of bits were precious and you really didn't want to use up more than necessary bits

That is why people tried to optimise the number of bits

- What about other characters? あ आ 아 𐄌
 - 1000s of characters needed

Unicode

- Allow codes for more script, characters
- How many?
 - All the living languages?
 - All extinct languages?
 - All future languages?
- **"Universal Character Set"** encoding - UCS
 - **UCS-2** 2 bytes per character - max 65,536 characters
 - **UCS-4** 4 bytes per character - 4B+ characters



Week 2 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@September 14, 2021 4:50 PM
🔗 Materials	
# Module #	11
▼ Type	Lecture
☰ Week #	2

Efficiency of encoding

Efficiency

- What is the most common language on the web?
 - As of now, it is English
- Should all the characters be represented with the same number of bits?
- Example:
 - A text document with 1,000 words, which equates to approximately 5,000 characters (including spaces)
 - According to UCS-4 encoding
$$32 \text{ bits} \times 5,000 = 160,000 \text{ bits}$$
 - According to ASCII encoding
$$8 \text{ bits} \times 5000 = 40,000 \text{ bits}$$
 - According to the original 7-bit ASCII which was sufficient for English
$$7 \text{ bits} \times 5000 = 35,000 \text{ bits}$$
 - Minimum needed to encode just 'a' - 'z', numbers and some special characters could fit in 6-bits: 30,000 bits
 - Optimal coding based on frequency of occurrence of a letter
 - 'e' is the most common letter, 't', 'a', 'o', ...
 - Huffman or similar encoding: ~10,000 - 20,000 bits, possibly less

Is this problem solvable in general?

- It is impossible to encode by actual character frequency: depends on the text
 - Just use compression methods like "zip" instead
- But can encoding be a good halfway point?

Example:

- Use 1-byte for most common alphabets
- Group others according to frequency, have "prefix" codes to indicate

Prefix Coding

 1st byte	 2nd byte	 3rd byte	 4th byte	 Free bits	 Max. expressible Unicode val.
<u>0</u> xxxxxxx				7	007F hex (127)
<u>110</u> xxxxx	10xxxxxx			(5+6)=11	07FF hex (2047)
<u>1110</u> xxx	10xxxxxx	10xxxxxx		(4+6+6)=16	FFFF hex (65535)
<u>11110</u> xx	10xxxxxx	10xxxxxx	10xxxxxx	(3+6+6+6)=21	10FFF hex (1,114,111)

- If the first bit starts with a 0, take the remaining 7 free bits as an ASCII code

Example

	A	ᄀ	好	丕
Code point	U+0041	U+05D0	U+597D	U+233B4
UTF-8	41	D7 90	E5 A5 BD	F0 A3 8E B4
UTF-16	00 41	05 D0	59 7D	D8 4C DF B4
UTF-32	00 00 00 41	00 00 05 D0	00 00 59 7D	00 02 33 B4

Source: <https://www.w3.org/International/articles/definitions-characters>

UTF-8

- Stands for Unicode Transformation Format
- Uses 8-bits for most common characters: ASCII subset
 - All ASCII documents are automatically UTF-8 compatible
- All other characters can be encoded based on prefix string
- More difficult for text processors:
 - first check prefix
 - linked list through chain of prefixes possible
 - Still more efficient for majority of documents
- Most common encoding in use today



Week 2 Lecture 3

▼ Class	BSCCS2003
🕒 Created	@September 14, 2021 6:39 PM
🔗 Materials	
# Module #	12
▼ Type	Lecture
☰ Week #	2

What is Markup?

Topics

- Content v/s Meaning
- Types of Markup
- (X)HTML

Raw content

Markup What is markup? Markup is a way of using cues or codes in the regular flow to indicate how text should be displayed. Markup is very useful to make the display of text clear and easy to understand

Types of Markup

- Presentational
 - **WYSIWYG**: directly format output and display
 - What You See Is What You Get
 - Embed codes not part of regular text, specific to the editor
- Procedural
 - Details on how to display something
 - Change the font to large, bold
 - skip 2 lines, indent 4 columns

- Descriptive
 - This is a `<title>`, this is a `<heading>`, this is a `<paragraph>`

Examples

- Microsoft Word, Google Docs, etc
 - User interface focused on "appearance", not meaning
 - WYSIWYG: direct control over styling
 - Often leads to complex formatting and loss of inherent meaning
- LaTeX, HTML (general *ML)
 - Focus on meaning
 - More complex to write and edit, not WYSIWYG in general
 - Fun fact: The current raw document was written in LaTeX and rendered to a PDF later

Semantic Markup

- Content v/s Presentation
- Semantics
 - Meaning of the text
 - Structure or Logic of the document



Week 2 Lecture 4

▼ Class	BSCCS2003
🕒 Created	@September 14, 2021 7:21 PM
🔗 Materials	
# Module #	13
▼ Type	Lecture
☰ Week #	2

Introduction to HTML

Topics

- HyperText Markup Language
- Generalizations
- Variants of Interest

HyperText Markup Language (HTML)

- HTML was first used by Tim Berners-Lee in the original Web at CERN (-1989)
- Considered an application of SGML (Standard Generalized Markup Language)
 - Strict definitions on structure, syntax and validity
- HTML was meant for browser interpretation
 - Very forgiving: loose validity checks
 - Best effort to display

HTML Example

```
<!DOCTYPE html>
<html>
<body>
<h1>My First Heading.</h1>
<p>My first paragraph.</p>
</body>
</html>
```

Tags

- `<h1></h1>` - paired tags
- Represented by angled brackets `<>`
- Closing tag with `/`
- Location specific: `<DOCTYPE>` is only at the head of the document
- Case insensitive

Nesting

Valid syntax

- `<em><strong>Hello</strong></em>`
- The output is
 - ***Hello***
- Invalid:
 - `<em><strong>Hello</em></strong>`
 - `<em><strong>Hello</em>`
 - `<em><strong>Hell<o/em></strong>`

Presentation v/s Semantics

- `<strong>Hello</strong>`
- `<b>Hello</b>`
- **Hello**

Which one is right? Which one is better?

It is not an easy question to answer, both of them may be correct

`<strong>` is putting an emphasis meanwhile `<b>` does not

Timelines

- SGML based
 - 1989 - HTML original
 - 1995 - HTML 2
 - 1997 - HTML 3, 4
- XML based
 - XHTML - 1997 - mid 2010s
- HTML 5
 - first release in 2008
 - W3C recommendation - 2014
 - World Wide Web Consortium

HTML 5

- Block element: `<div>`
- In-line element: `<span>`
- Logical elements: `<nav>`, `<footer>`
- Media: `<audio>`, `<video>`

Remove "presentation only" tags:

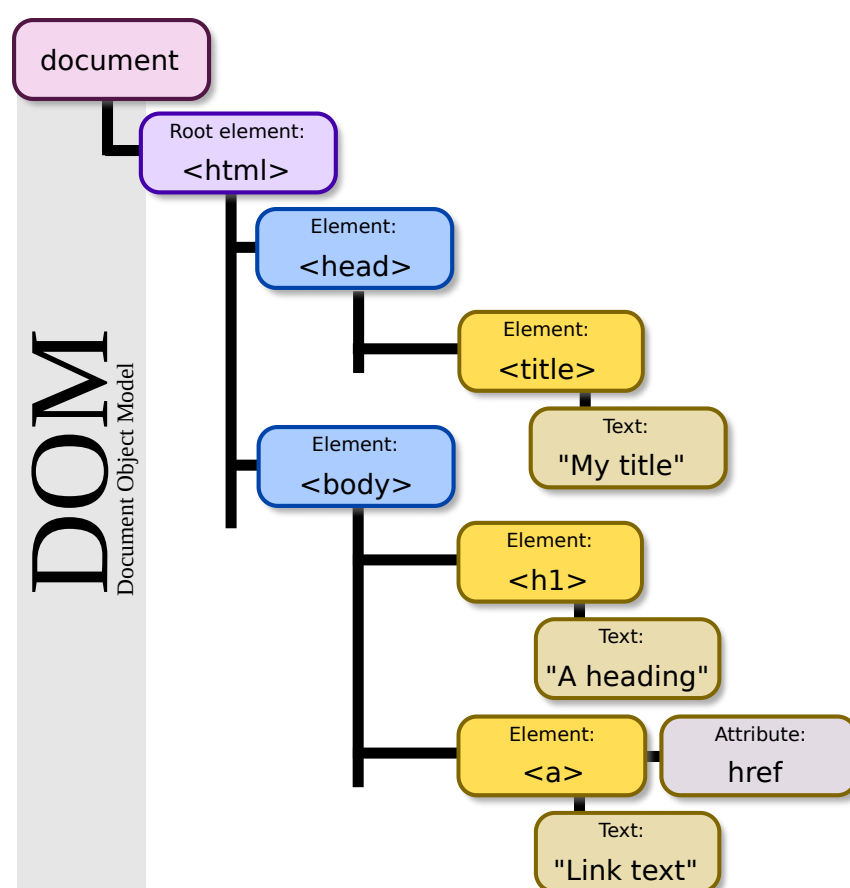
- `<center>`
- `<font>`

Document Object Model (DOM)

The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. The DOM represents the document as nodes and objects; that way, programming languages can interact with the page.

Source: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

```
<html>
<head>
  <title>My title</title>
</head>
<body>
  <h1>A heading</h1>
  <a href="https://onlinedegree.iitm.ac.in">Link text</a>
</body>
</html>
```



Source: https://en.wikipedia.org/wiki/Document_Object_Model

- It is a tree structure which represents the logical layout of the document
- Direct manipulation of this tree is possible!
- Application Programming Interfaces (APIs) are also defined on the DOM:
 - Canvas
 - Offline
 - Web Storage
 - Drag & Drop
 - ...
- In most of the cases, when we talk about manipulating the DOM, we use JavaScript
- Cascading Style Sheets (CSS) is used for styling



Week 2 Lecture 5

▼ Class	BSCCS2003
🕒 Created	@September 15, 2021 12:59 PM
🔗 Materials	
# Module #	14
▼ Type	Lecture
☰ Week #	2

Introduction to Styling

Topics

- Markup v/s Style
- Themes
- CSS

Markup v/s Style

```
<h1>Hello</h1>
```

<https://codepen.io/21f1003586/pen/bGRoJJX>

<https://codepen.io/21f1003586/pen/BaZweBX>

Separation of Styling

- Style hints in separate blocks
 - Separate files included
- Themes

- Style sheets
 - Specify presentation information
- Cascading Style Sheets (CSS)
 - Allows multiple definition
 - Latest definition takes the precedence



Week 2 Lecture 6

▼ Class	BSCCS2003
🕒 Created	@September 15, 2021 2:23 PM
🔗 Materials	
# Module #	15
▼ Type	Lecture
☰ Week #	2

Types of CSS styling & Responsive Websites

Inline CSS

- Directly add style to the tag
- Example:

```
<h1 style="color: blue; text-align: center;">A heading</h1>
```

Internal CSS

- Embed inside the `<head>` tag
- Now, all the `<h1>` tags in the document will look the same - centrally modified

```
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
```

External CSS

- Extract common content for re-use

- Multiple CSS files can be included
- Latest definition of style takes the precedence

Responsive Design

- Mobile and Tablet devices have smaller screen
 - Different form factors
- Adapt to the screen - ***Respond***
- CSS controls the styling - HTML controls the client

Bootstrap

- Most commonly used framework for CSS
 - Originate from Twitter
 - Widely used now
- Standard styles for various components
 - Buttons
 - Forms
 - Icons
 - ...
- Mobile first: highly responsive layout

JavaScript

- It is an interpreted language brought into the browser
- Not really related to Java in any way - formally known as ECMAScript
- Why was JavaScript brought into the browser?
 - HTML is not a programming language
 - CSS is not a programming language too (There are ways to make it Turing complete, google it)
- We wanted to have "programmability" inside the browser
- It is not a part of the core presentation requirements
 - But still, it is very useful and will be considered later

Summary

- Presentation - Human interaction
- Separate content from style
 - Markup - HTML
 - Styling - CSS



Week 3 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 5:06 PM
🔗 Materials	
# Module #	16
▼ Type	Lecture
☰ Week #	3

Overview of MVC

Outline

- MVC paradigm
- Views and User interfaces
- Tools & Techniques
- Accessibility

Model: Store e-mails on the server; indexed and ready to manipulate

View: Display a list of e-mails; Read individual e-mail

Controller: Sort e-mails; delete; archive (Usually the business logic of an app)

Model-View-Controller

- Origins: Smalltalk-80
- Separation of responsibilities - **Abstraction**
- Roots in Object-Oriented GUI development

Design Patterns

- Common software patterns
- **Model:** Application object
- **View:** Screen representation
- **Controller:** How does the user interface reacts to user input

Running Example

Student Gradebook

- Input data: for **Model**
 - Student list
 - Course list
 - Student-Course marks
- Outputs: for **View**
 - Marks for individual student
 - Summary of the course
 - Histograms
- Modifications: for **Controller**
 - Add new students
 - Add new courses
 - Modify the marks in a course



Week 3 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 5:20 PM
🔗 Materials	
# Module #	17
▼ Type	Lecture
☰ Week #	3

Views

User Interface Design

User Interface

- Screen
- Audio
- Vibration (haptics)
- Motor (door open / close)

User Interaction

- Keyboard / Mouse
- Touchscreen
- Spoken voice
- Custom buttons

User interaction is ...

- determined by the hardware constraints
- different based on various types of devices
- user-agent information useful to identify context

Types of Views

- Fully static
- Partly dynamic
- Mostly dynamic

Output

- HTML - most commonly used - direct rendering
- Dynamic images
- JSON / XML - machine readable

View - any "representation" that is useful to another entity



Week 3 Lecture 3

▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 5:37 PM
🔗 Materials	
# Module #	18
▼ Type	Lecture
☰ Week #	3

User Interface Design

- Design for interaction with the user
- Goals:
 - **Simple:** Easy for the user to understand and use
 - **Efficient:** User achieves their goal with minimal effort
- Aesthetics
 - Something which is pleasing to the eyes and looks nice
- Accessibility

Systematic Process

- Functionality requirements gathering - What is needed?
- User and Task analysis - user preferences, task needs
- Prototyping - wireframes, mock-ups
- Testing - User acceptance, usability, accessibility



Week 3 Lecture 4

▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 6:27 PM
🔗 Materials	
# Module #	19
▼ Type	Lecture
☰ Week #	3

Usability Heuristics

Guidelines / Heuristics

Jakob Nielsen's heuristics for design

<https://www.nngroup.com/articles/ten-usability-heuristics/>

- Not specific to web apps, or even software UI design
- But it is very useful and relevant

General principles

- Consistency
- Simple and minimal steps
- Simple language
- Minimal and aesthetically pleasing



Week 3 Lecture 5

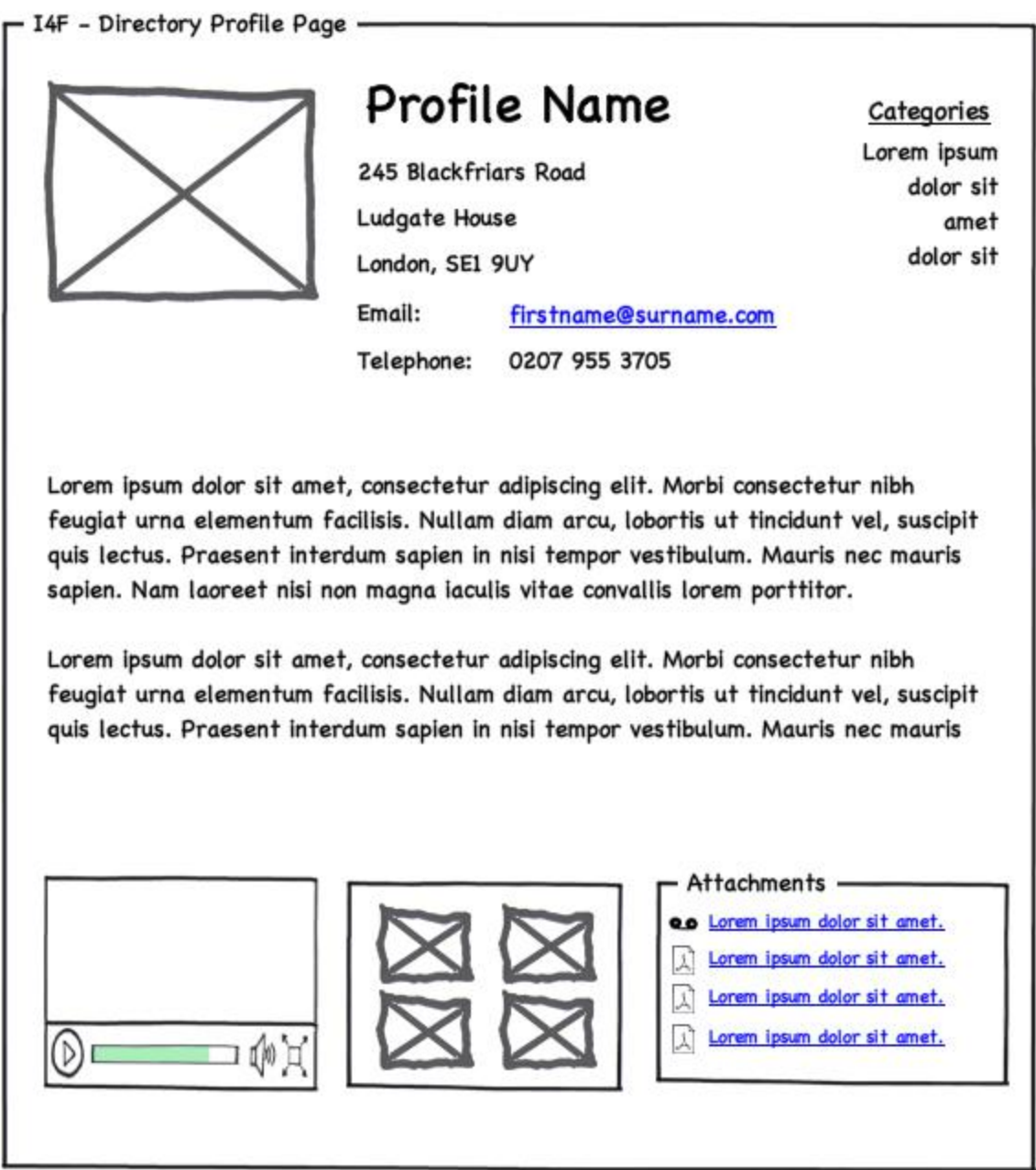
▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 6:36 PM
🔗 Materials	
# Module #	20
▼ Type	Lecture
☰ Week #	3

Tools (part 1)

- Wireframes
- HTML generation
- Templates

Wireframes

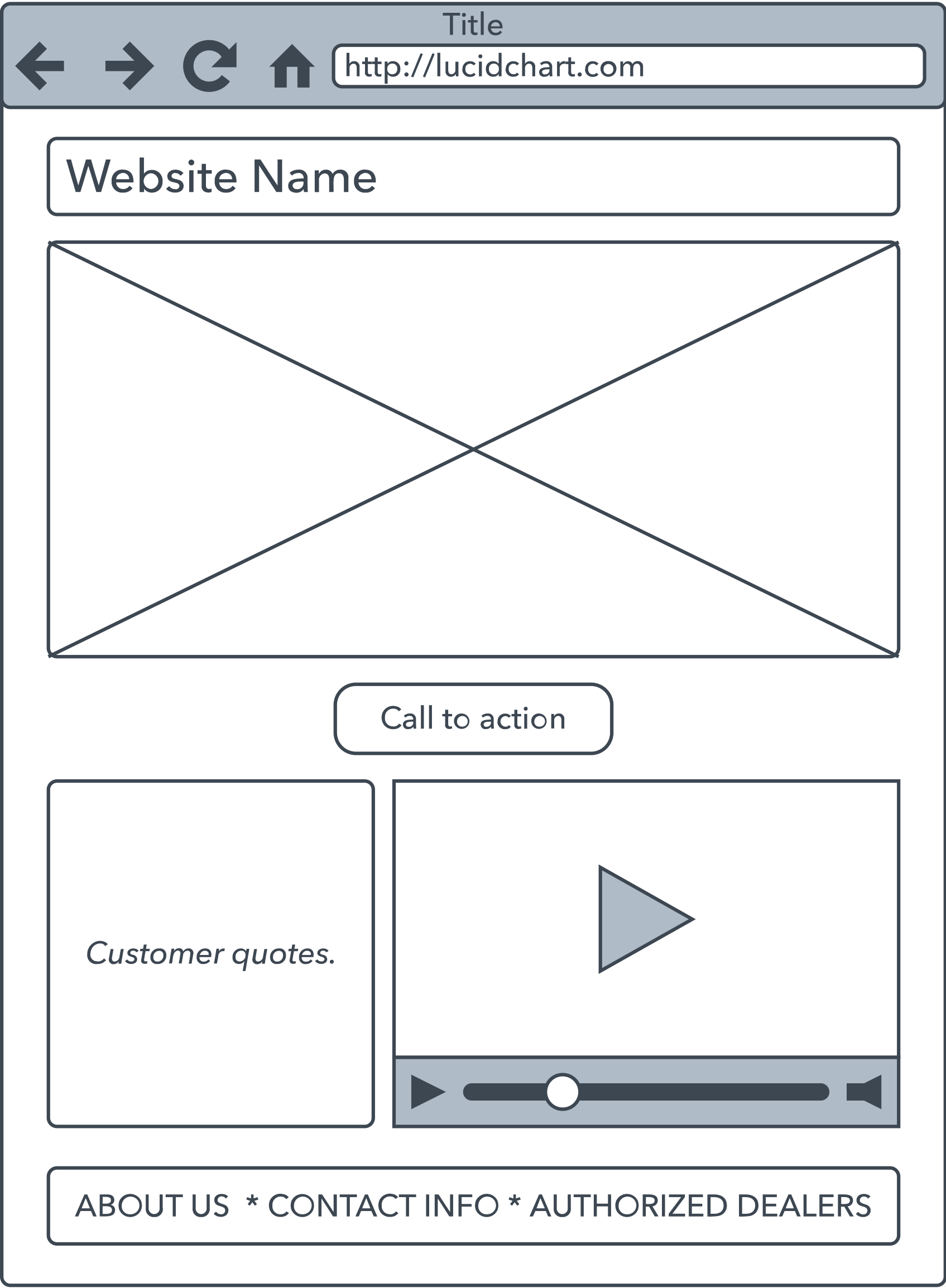
- Visual guide to represent **structure** of the web page
- Information design
- Navigation design
- User interface design



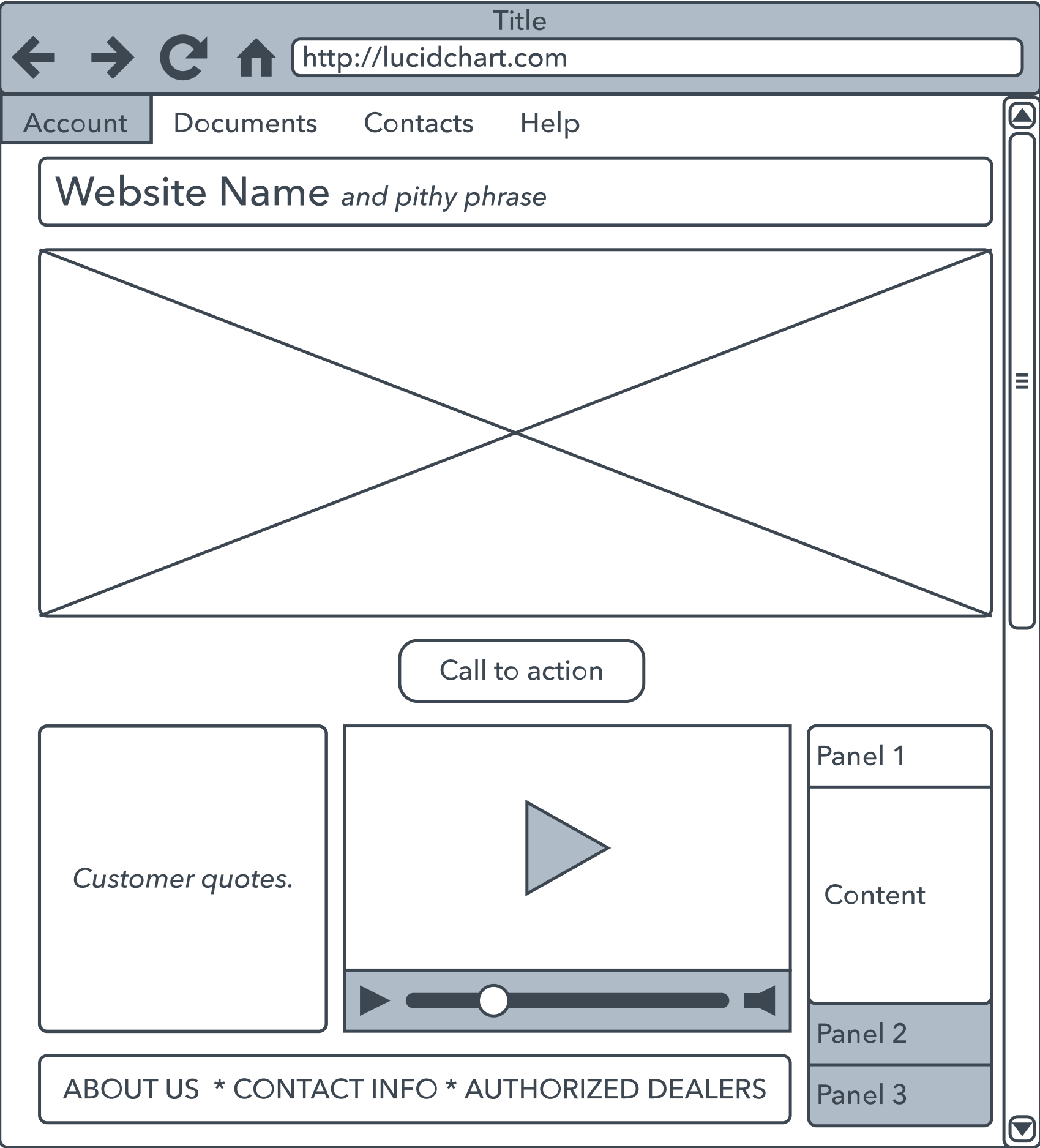
created with Balsamiq Mockups - www.balsamiq.com

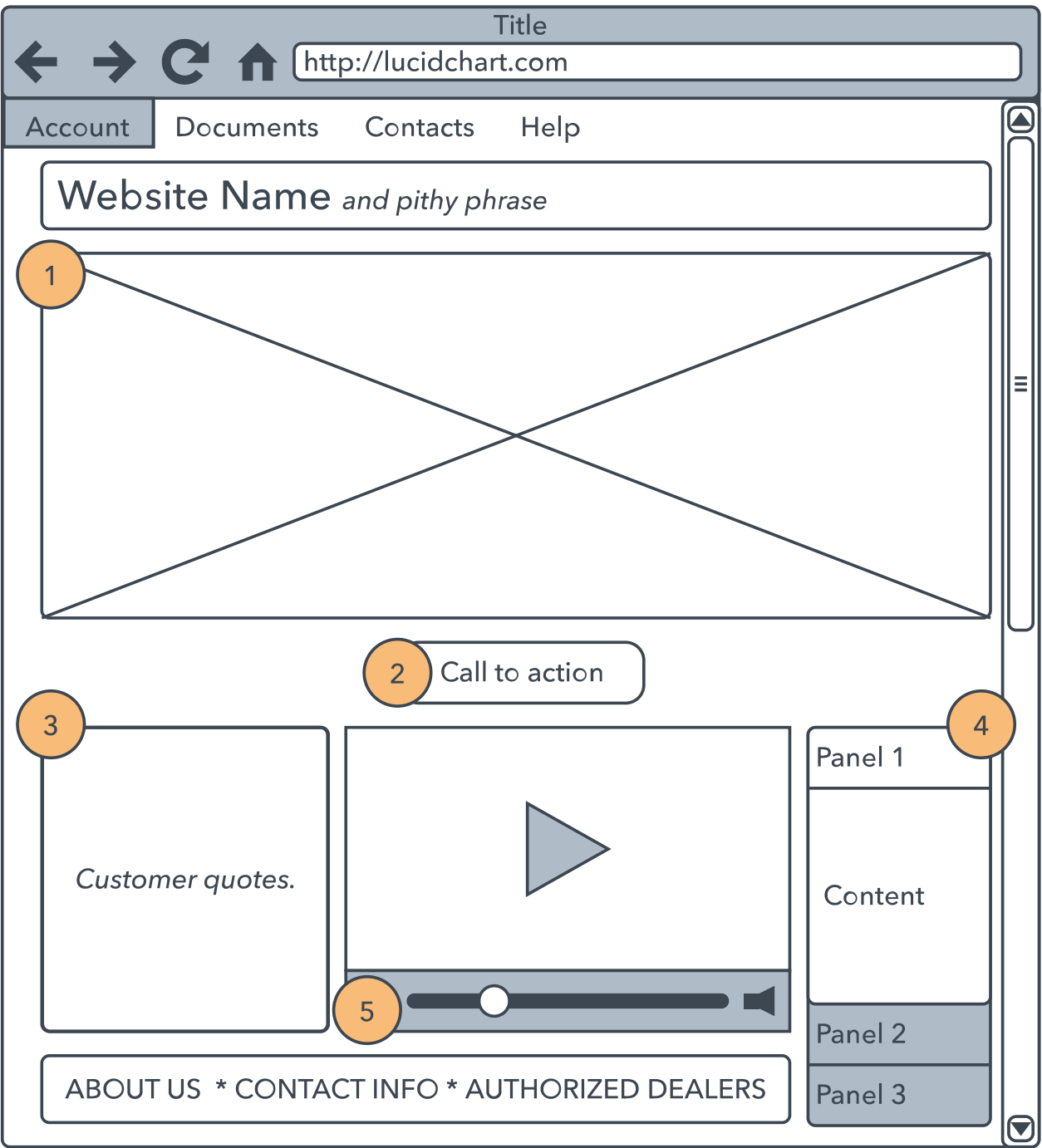
Source: https://en.wikipedia.org/wiki/Website_wireframe

Example Tools: LucidCharts



Source: <https://www.lucidchart.com/pages/wireframe>





Notes

- 1 Details about the image
- 2 Where this links to
- 3 List of possible quotes
- 4 Drop down menus that link to other pages
- 5 Content of video



Week 3 Lecture 6

▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 6:47 PM
🔗 Materials	
# Module #	21
▼ Type	Lecture
☰ Week #	3

Tools (part 2)

Programmatic HTML generation: PyHTML

- Composable functions - each function generates a specific output
- Example:
 - To generate an `h1` heading: the function should return ..

```
"<h1>text of heading</h1>"
```

```
import pyhtml as h

t = h.html(
    h.head(
        h.title('Test page')
    ),
    h.body(
        h.h1('This is a title'),
        h.div('This is some text'),
        h.div(h.h2('inside title'),
            h.p('some text in paragraph'))
    )
)
print(t.render())
```

More complex HTML

```
def f_table(ctx):
    return (
        tr(
            td(cell) for cell in row
```



```
) for row in ctx['table']
)
```

Templates

- Standard template text
- Placeholder / Variables
- Basic (very limited) programmability
- Examples:
 - Python inbuilt string templates - good for simple tasks
 - Jinja2 - used by Flask
 - Genshi
 - Mako
 - ...

Jinja

- Ties in closely with Flask
- Template functionality with detailed API

Remember: Templates can generate any output, not just HTML

```
from string import Template

t = Template('$name is the $job of the $company')
s = t.substitute(name='Tim Cook', job='CEO', company='Apple Inc.')
print(s)
```

Jinja example

```
from jinja2 import Template
t = Template('Hello {{ something }}!')
print(t.render(something='world'))

t = Template('My favourite numbers: {% for n in range(1, 10) %}{{ n }} ' '{% endfor %}')
```



Week 3 Lecture 7

▼ Class	BSCCS2003
🕒 Created	@September 18, 2021 8:25 PM
🔗 Materials	
# Module #	22
▼ Type	Lecture
☰ Week #	3

Accessibility

- Various forms of disability or impairment
 - Vision
 - Speech
 - Touch
 - Sensor-Motor
- Can a page be accessed by people with impairments?
- How can the accessibility of a page be improved?

[W3.org](#) - World Wide Web Consortium (W3C) - Accessibility guides

<https://www.w3.org/WAI/fundamentals/accessibility-principles>

Standards

Interplay between many components of a page:

- Web content: HTML, images, scripts, etc
- User-agents: Desktop browser, Mobile browser, speech-oriented browser, assistive devices
- Authoring tools: Text editor, Word processors, Compiler

Principle - Preceivable

- Provide text alternatives for non-text content
- Provide captions and other alternatives for multimedia

- Create content that can be presented in different ways, including by assistive technologies, without losing meaning.
- Make it easier for users to see and hear content

Principle - Operation

- Make all functionality available from a keyboard
- Give users enough time to read and use content
- Do not use content that causes seizures or physical reactions
- Help users navigate and find content
- Make it easier to use inputs other than keyboard

Principle - Understandable

- Make the text readable and understandable
- Make the content appear and operate in predictable ways
- Help users avoid and correct mistakes

Principle - Robust

- Maximize compatibility with current and future user tools

Aesthetics

- Visual appearance
- Very important
- Simplicity preferred

Can vary with time



Source:

iOS: A visual history

In what is widely regarded as his greatest presentation ever, Apple's Steve Jobs introduced the iPhone to the world on January 9th, 2007. In the five-plus years since then, the iPhone, iPad, and iPod Touch have literally redefined the entire world of mobile computing.

▼ <https://www.theverge.com/2011/12/13/2612736/ios-history-iphone-ipad>



Summary

- View - any screen seen by the human or machines
- User-interface and User-interaction guidelines
- Accessibility is a core concept
- Tools for automatic generation, consistent layout



Week 4 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@September 27, 2021 1:48 PM
🔗 Materials	
# Module #	23
▼ Type	Lecture
☰ Week #	4

Persistent Storage

Example: Gradebook

- Students: ID, name, address, ...
- Courses: ID, name, department, year, ...
- Student-Course Relationship: which students are registered for which courses

Gradebook

Aa Name	☰ IDNumber
<u>Sunil Shashi</u>	MAD001
<u>Chetana Anantha</u>	MAD002
<u>Madhur Prakash</u>	MAD003
<u>Nihal Surya</u>	MAD004
<u>Shweta Lalita</u>	MAD005
<u>Raghu Balwinder</u>	MAD006
<u>Gulshan Kuldeep</u>	MAD007
<u>Kishan Shrivatsa</u>	MAD008
<u>Purnima Sunil</u>	MAD009
<u>Nikitha Madhavi</u>	MAD010
<u>Lilavati Prabhakar</u>	MAD011
<u>Rama Yamuna</u>	MAD012

Gradebook

Aa CourseID	☰ Name
<u>EE1001</u>	Introduction to Electrical Engineering
<u>AM1100</u>	Engineering Mechanics
<u>MA1020</u>	Functions of Several Variables
<u>ME1100</u>	Thermodynamics
<u>BR1010</u>	Life Sciences

Spreadsheets

- Arbitrary data organized into rows and columns
- Operations defined on cells or ranges
- Multiple inter-linked sheets within single spreadsheet

Any kind of tabular data - expressed in tables

Relationships

- Students - Course?
- Separate entry with full details - student name, ID, address, course ID, name, department, etc.
 - Redundant
- Separate table "joining" students with courses
 - Only ID specified
 - Relation specified with "Keys"

<u>Aa</u> A	<u>≡</u> B	<u>#</u> C
<u>MAD001</u>	BT1010	78
<u>MAD002</u>	EE1001	30
<u>MAD005</u>	EE1001	68
<u>MAD009</u>	AM1100	62
<u>MAD012</u>	AM1100	77
<u>MAD007</u>	BT1010	41

Questions

- How should the underlying data should be stored?
 - Can it be made persistent - survive server restart?
- How should the relations be represented?
- Structured ways to represent, manipulate data?



Week 4 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@September 27, 2021 2:41 PM
🔗 Materials	
# Module #	24
▼ Type	Lecture
☰ Week #	4

Mechanisms for Persistent Storage

In-memory data structures

```
names = ['Alice', 'Bob', 'Charlie']
courses = ['Introduction to EE', 'Applied Mech', 'Calculus']
rels = [('Alice', 'Introduction to EE'),
        ('Bob', 'Calculus'),
        ('Alice', 'Calculus'),
        ('Charlie', 'Applied Mech')]
```

- Error prone - easy to make mistakes in entry or referencing
- Does not scale
- Duplicate names?

In-memory data structures: Keys

```
names = {0: 'Alice', 1: 'Bob', 2: 'Charlie'}
courses = {0: 'Introduction to EE', 1: 'Applied Mech', 2: 'Calculus'}
rels = [(0, 0), (1, 2), (0, 2), (2, 1)]
```

- Data entry errors less likely
- Duplicates not a problem - Unique key

Objects

```
class Student:
    id_next = 0 # Class variable
    def __init__(self, name):
        self.name = name
        self.id = Student.id_next
        Student.id_next += 1
```

- Auto-initialize ID to ensure unique
- Functions to set/get values

```
class Student:
    id_next = 0 # Class variable
    def __init__(self, name, hostel):
        self.name = name
        self.id = Student.id_next
        self.hostel = hostel
        Student.id_next += 1
```

- Add a new field to the object easily

But what about persistence?

- In-memory data structures are lost when the program is killed, server is shut down or rebooted (restarted)
- Save to disk? Structured data?
 - Python Pickle and similar modules
 - CSV - Comma Separated Values
 - TSV - Tab Separated Values
- Essentially the same as spreadsheets - limited flexibility

Spreadsheets

- Naturally represent tabular data
- Extension, adding fields are easy
- Separate sheets for relationships

Problems with spreadsheets:

- Lookups, cross-referencing harder than dedicated database
- Stored procedures - limited functionality
- Atomic operations - no clear definition

Relational Databases - SQL

Shameless plug, I do have notes for DBMS here

- From IBM ~ 1970s
- Data is stored in tabular format:
 - Columns of tables: fields (name, address, department, ...)
 - Rows of tables: individual entries (student1, student2, ...)

Unstructured Databases - NoSQL

- Easily add/change fields
- Arbitrary data
- NoSQL
 - MongoDB
 - CouchDB
 - ...

- Flexible, but potential loss of validation



Week 4 Lecture 3

▼ Class	BSCCS2003
🕒 Created	@September 27, 2021 3:39 PM
🔗 Materials	
# Module #	25
▼ Type	Lecture
☰ Week #	4

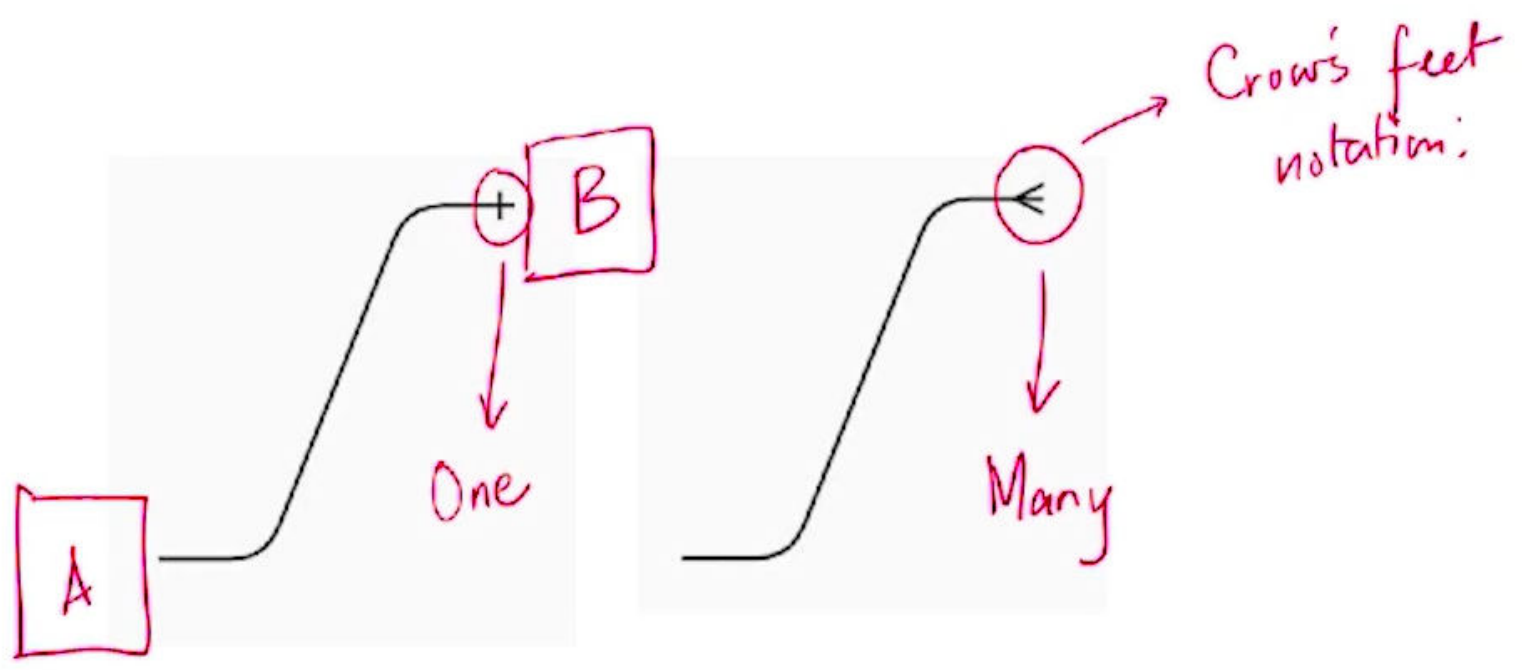
Relations and ER diagram

Relationship types

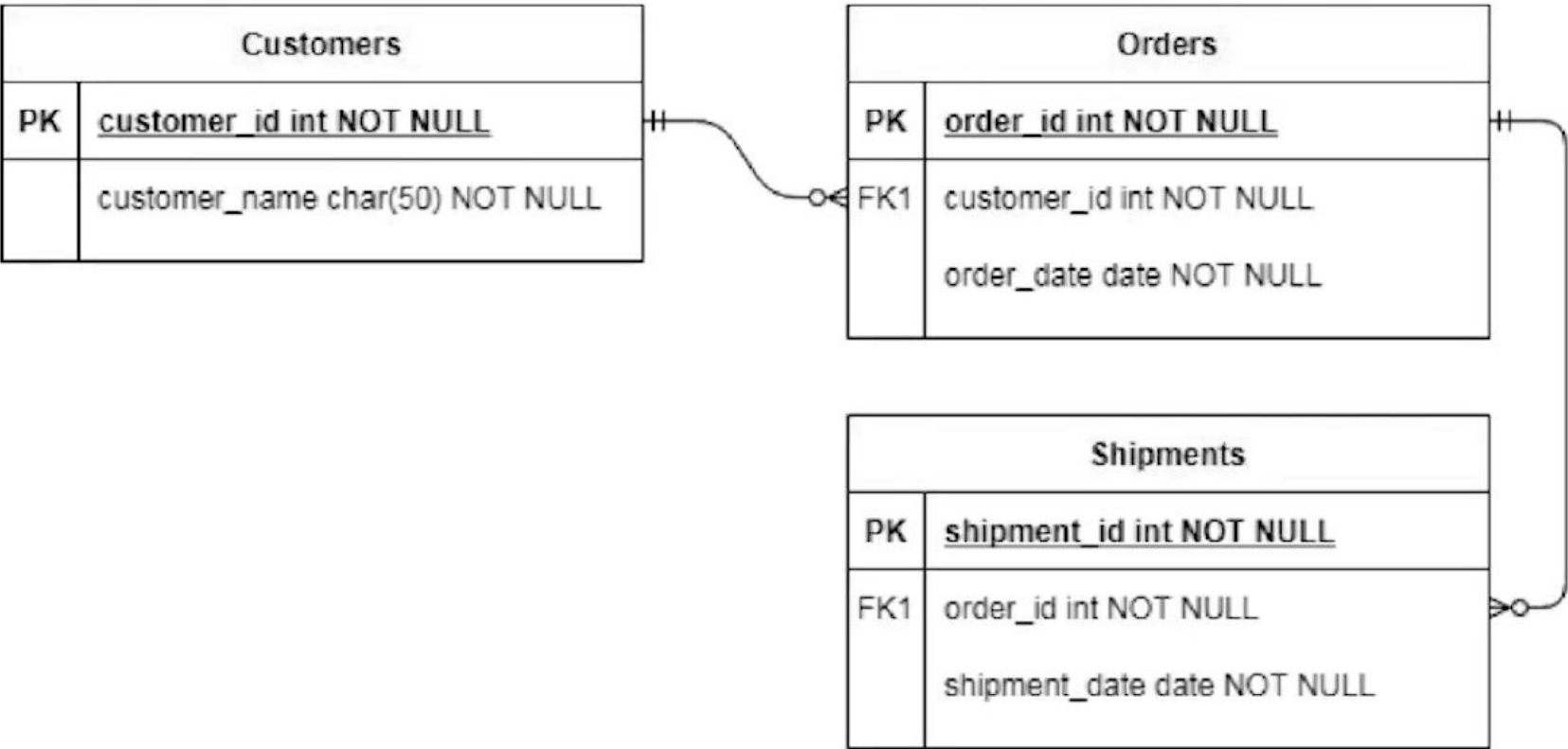
- One to One:
 - One student has one roll number
 - One roll number uniquely identifies one student
 - **Example:** assign unique message-ID to each email in the inbox
- One to Many (Many to One):
 - One stays in only one hostel
 - One hostel has many students
 - **Example:** save emails in folders - one email is in only one folder
- Many to Many:
 - One student can register for many courses
 - One course be taken by many students
 - **Example:** assign labels to emails - one email can have many labels and vice-versa

Diagrams

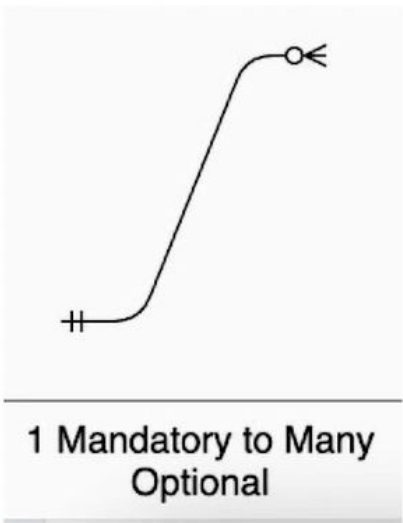
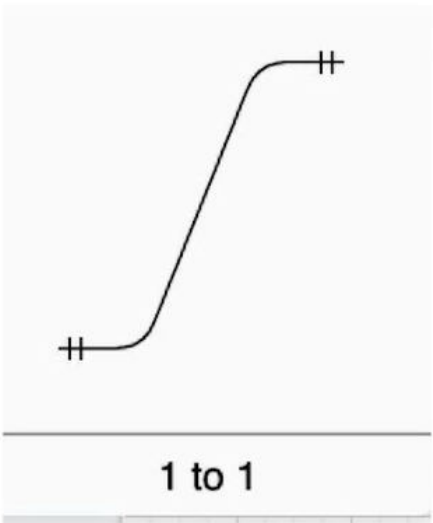
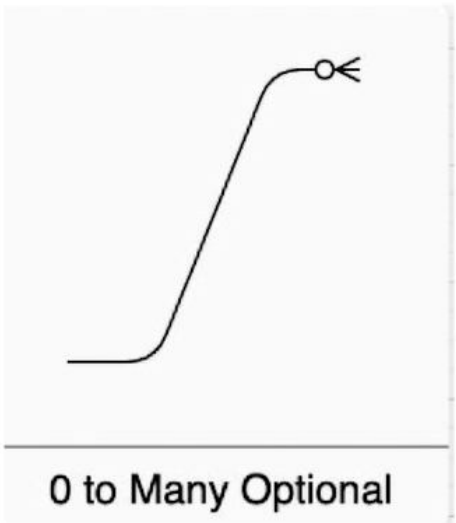
- Entity-Relationship (ER)
- Unified Modeling Language (UML)
- Class relation



Entity-Relationship Diagram



ER examples



Tool: Draw.io → <https://app.diagrams.net>



Week 4 Lecture 4

▼ Class	BSCCS2003
🕒 Created	@September 27, 2021 4:29 PM
🔗 Materials	
# Module #	26
▼ Type	Lecture
☰ Week #	4

SQL

Relational Databases - SQL

- From IBM ~ 1970s
- Data is stored in tabular format:
 - Columns of tables: fields (name, address, department, ...)
 - Rows of tables: individual entries (student1, student2, ...)
- Key: unique way of accessing a given row
 - **Primary Key**: important for fast access to large databases
 - **Foreign Key**: connect to a different table - Relationships

Queries

- Retrieve data from a database

eg: "Find students with name beginning with A"
"Find all courses offered in 2021"

Structured Query Language

- English-like, but structured
- Quite verbose
- Specific mathematical operations:
 - Inner join

- Outer join

Example: Inner join

<u>Aa</u> Name	<u>≡</u> IDNumber	<u>#</u> hostelID
<u>Sunil Shashi</u>	MAD001	1
<u>Chetana Anantha</u>	MAD002	2
<u>Madhur Prakash</u>	MAD003	2
<u>Nihal Surya</u>	MAD004	3
<u>Shweta Lalita</u>	MAD005	2
<u>Raghu Balwinder</u>	MAD006	3
<u>Gulshan Kuldeep</u>	MAD007	1
<u>Kishan Shrivatsa</u>	MAD008	1
<u>Purnima Sunil</u>	MAD009	2
<u>Nikitha Madhavi</u>	MAD010	1
<u>Lilavati Prabhakar</u>	MAD011	3
<u>Rama Yamuna</u>	MAD012	3

<u>Aa</u> ID	<u>≡</u> Name	<u>#</u> Capacity
<u>1</u>	Jamuna	300
<u>2</u>	Ganga	300
<u>3</u>	Brahmaputra	500

Student - Hostel mapping

```
SELECT Students.name, Hostels.name
FROM Students
INNER JOIN Hostels
ON Students.hostelID = Hostels.ID;
```

```
---- OUTPUT ----
Sunil Shashi, Jamuna
Chetana Anantha, Ganga
```

Cartesian product

- N entries in table 1
- M entries in table 2
- $M \times N$ combinations - filter on them

Powerful SQL queries can be constructed

Example: find all the students in Calculus

- Find ID number for the course
- Look up StudentsCourses table to find all entries with this course ID
- Look up Students to find the names of students with these IDs

```
SELECT s.name
FROM Student s
JOIN StudentsCourses sc ON s.IDNumber = sc.studentID
JOIN Courses c ON c.ID = sc.courseID
WHERE c.name = 'Calculus';
```

Summary

- Models - persistent data storage
- Mechanisms:
 - CSV
 - spreadsheets
 - SQL
 - NoSQL

- Entities and Relationships
 - Different ways of representing

No details on display, views or what kind of updates permitted



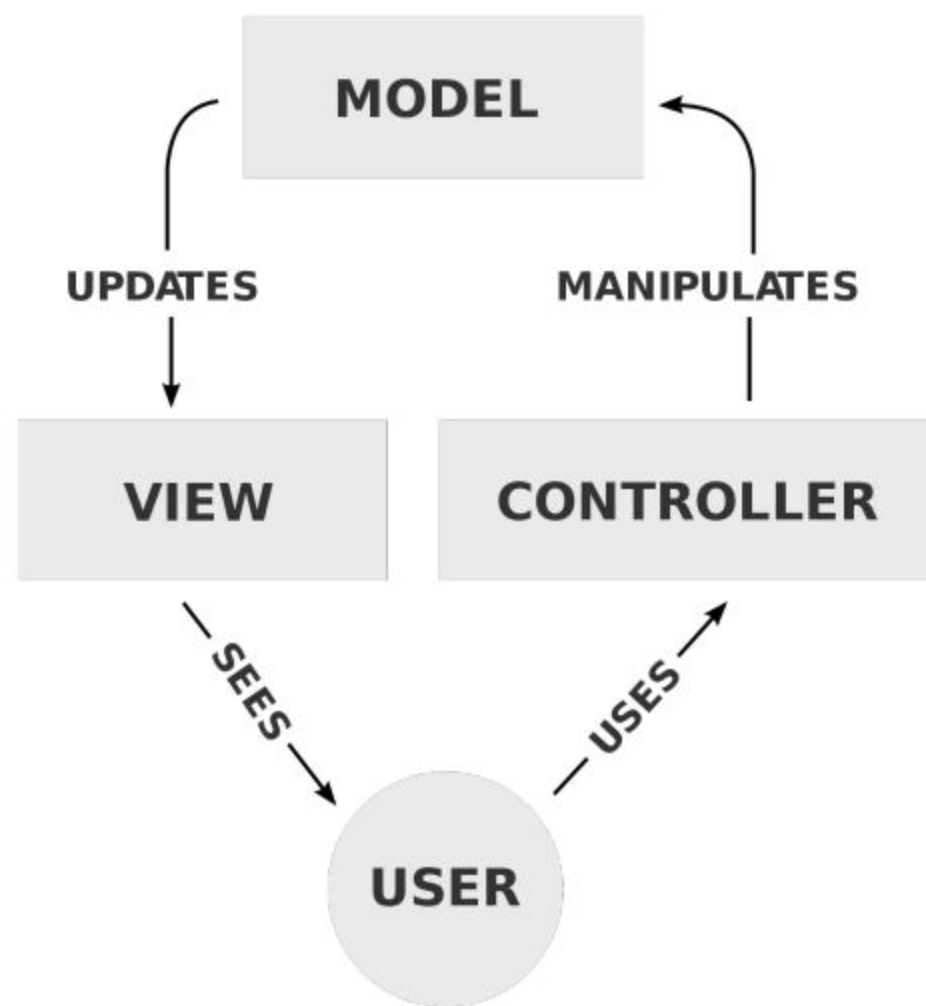
Week 5 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@October 4, 2021 12:18 AM
🔗 Materials	
# Module #	27
▼ Type	Lecture
☰ Week #	5

MVC Origins

Model-View-Controller

- Design pattern: or collection of design patterns
- Originally introduced in the context of GUI design in Smalltalk-80
- Many different variants, interpretations, ...



By RegisFrey - Own work, Public Domain,
Wikipedia

From: Trygve Reenskaug

Date: 10 December 1979

MODELS-VIEWS-CONTROLLERS

MODELS

Models represent knowledge. A model could be a single object (rather uninteresting), or it could be some structure of objects. ~~The proposed implementation supports knowledge represented in something resembling semantic nets (If I understand Laura correctly)~~

VIEWS

A view is a (visual) representation of its model. It would ordinarily highlight certain attributes of the model and suppress others. It is thus acting as a *presentation filter*.

A view is attached to its model (or model part) and gets the data necessary for the presentation from the model by asking questions. It may also update the model by sending appropriate messages. All these questions and messages have to be in the terminology of the model, the view will therefore have to know the semantics of the attributes of the model it represents. (It may, for example, ask for the model's identifier and expect an instance of Text, it may not assume that the model is of class Text)

CONTROLLERS

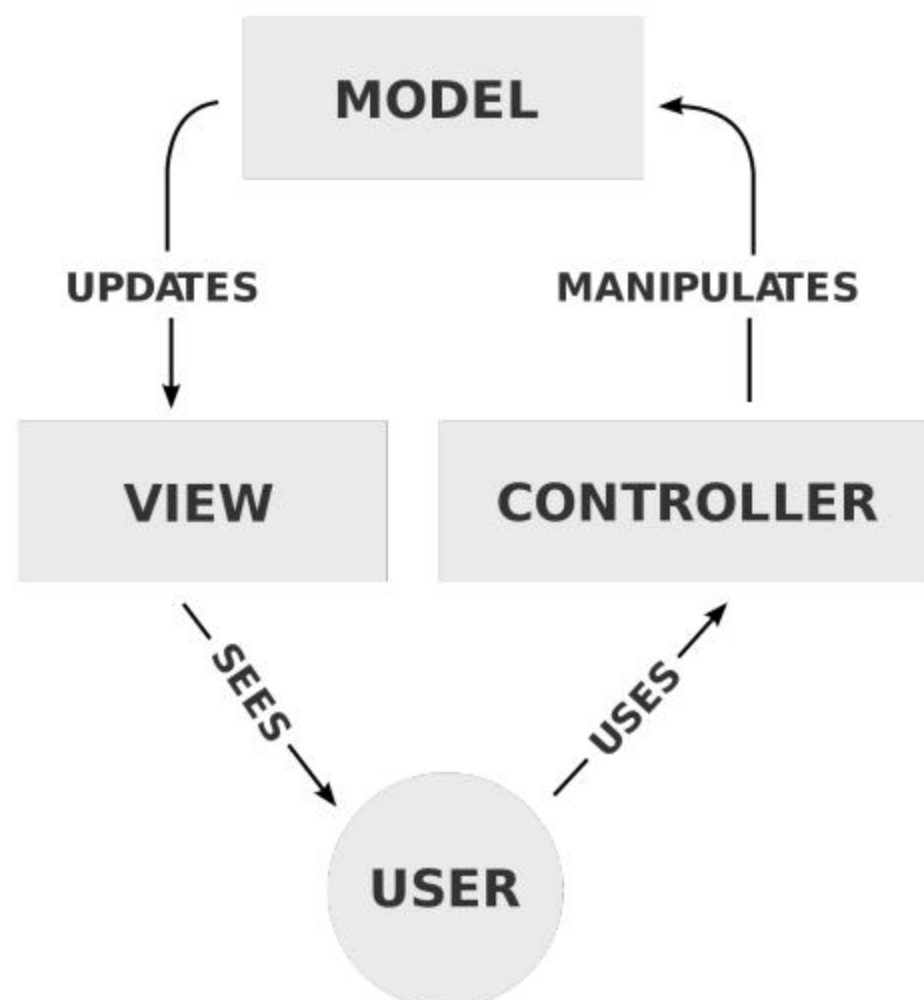
A controller is the link between a user and the system. It provides the user with input by arranging for relevant views to present themselves in appropriate places on the screen. It provides means for user output by presenting the user with menus or other means of giving commands and data. The controller receives such user output, translates it into the appropriate messages and pass these messages on .to one or more of the views.

A controller should never supplement the views, it should for example never connect the views of nodes by drawing arrows between them.

Conversely, a view should never know about user input, such as mouse operations and keystrokes. It should always be possible to write a method in a controller that sends messages to views which exactly reproduce any sequence of user commands.

General Concept - Action

- Take action in response to user input
- Communicate with the model, extract the view



Applicability

- Originally designed for GUI applications
- Separation of concerns - model vs views - and connection through controller
- State of interaction maintained as part of overall system memory

Web?

- Server does not maintain the state of the client
- Client is pure front-end to the user
- Some of the analogies break down - hence many variants of MVC

Present Context

- MVC is a good conceptual framework to understand separation of concerns
- Breaks down if applied too rigidly
- The web in general does not have the close knit structure of GUI applications needed for MVC
- Other aspects of static typing and type inference of objects also broken in Python-like languages

Apply basic learnings from MVC, but be prepared to stretch



Week 5 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@October 4, 2021 12:41 AM
🔗 Materials	
# Module #	28
▼ Type	Lecture
☰ Week #	5

Requests and Responses

Example dynamic web page

- View: page
- Links: clickable to select various options
- Clicking a link triggers different behaviors

Request - Response

- Web is based completely on requests and responses
 - Client makes requests
 - Server sends responses
- Basic requests: clicking on link / URL
 - HTTP GET
- More complex requests: form submissions
 - HTTP POST

Constraints?

- Any "page" can be requested
- Assignments, quizzes, lectures, general information

Are there common threads?

Example: Gradebook

- Students: ID, name, address, ...
- Courses: ID, name, department, year, ...
- StudentCourse Relationship: which students are registered for which courses

	A	B		A	B
1	Name	IDNumber	1	CourseID	Name
2	Sunil Shashi	MAD001	2	EE1001	Introduction to Electrical Engineering
3	Chetana Anantha	MAD002	3	AM1100	Engineering Mechanics
4	Madhur Prakash	MAD003	4	MA1020	Functions of Several Variables
5	Nihal Surya	MAD004	5	ME1100	Thermodynamics
6	Shweta Lalita	MAD005	6	BT1010	Life Sciences
7	Raghu Balwinder	MAD006			
8	Gulshan Kuldeep	MAD007			
9	Kishan Shrivatsa	MAD008			
10	Purnima Sunil	MAD009			
11	Nikitha Madhavi	MAD010			
12	Lilavati Prabhakar	MAD011			
13	Rama Yamuna	MAD012			

Common Operations

- Create a new student - add name, roll number, date of birth, ...
- Create a new course
- Assign a student to course
- Enter marks of student / Update marks of a student
- View summaries / charts / histograms
- Archive an old course
- Remove graduated students

Some essential functions can be distilled



Week 5 Lecture 3

▼ Class	BSCCS2003
🕒 Created	@October 4, 2021 12:51 AM
🔗 Materials	
# Module #	29
▼ Type	Lecture
☰ Week #	5

CRUD

Types of operations - Create

- Create a new entry
- Must not already exist
- Check within database to avoid conflicts
- Mention mandatory vs optional fields (name, address, mobile number) ...

Types of operations - Read

- Get a list of students
- Summarize number of students, age distribution, geographic locations
- Plot histograms of marks

Types of operations - Update

- Change of address
- Update marks
- Change start date of course

Type of operations - Delete

- Remove graduated students
- Delete mistaken entries

- Un-enroll students from course

CRUD

Create - Read - Update - Delete

- Originally in context of database operations: nothing to do with the Web
- Reflects cycle of data models
- Databases optimized for various combinations of operations
 - Read-heavy: lots of reading, very little writing or creating
 - Write-heavy: security archive logs

API - Application Programming Interface

- Standardized way to communicate with a server
- Client only needs to know the API - now how the server implements the database, for example
- CRUD is good set of functionality for a basic API
 - Usually considered the first level API to implement a web application
- Deals only with the data model life cycle - other control aspects possible



Week 5 Lecture 4

▼ Class	BSCCS2003
🕒 Created	@October 4, 2021 12:59 AM
🔗 Materials	
# Module #	30
▼ Type	Lecture
☰ Week #	5

Group Actions by Controller

Actions vs Controllers?

- CRUD etc. are a set of actions
- Other actions:
 - send email
 - update logs
 - send alert on WhatsApp / Telegram
- Can actions be grouped together logically?

Controller!

Example: Laravel PHP framework

Actions Handled By Resource Controller

Verb	URI	Action	Route Name
GET	/photos	index	photos.index
GET	/photos/create	create	photos.create
POST	/photos	store	photos.store
GET	/photos/{photo}	show	photos.show
GET	/photos/{photo}/edit	edit	photos.edit
PUT/PATCH	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Summary

- Actions: Interaction between view and model
- Controller: Group actions together logically
- API: Complex set of capabilities of server
- Interaction through HTTP requests
- HTTP verbs used to convey meanings

Rules of Thumb

- Should be possible to change views without the model ever knowing
- Should be possible to change the underlying storage of model without views ever knowing
- Controllers / Actions should generally NEVER talk to a database directly

In practice:

- Views and controllers tend to be more closely inter-linked than with Models
 - More about a way of thinking that a specific rule of design



Week 5 Lecture 5

▼ Class	BSCCS2003
🕒 Created	@October 4, 2021 1:13 AM
🔗 Materials	
# Module #	31
▼ Type	Lecture
☰ Week #	5

Routes and Controllers

Web applications

- Client-Server model
- Stateless: Server does not know the present state of the client
 - Must be ready to respond to whatever the client requests without assuming anything about the client
- Requests sent through HTTP protocol
 - Use variants of the GET, POST (verbs) to convey meaning
 - Use URL (Uniform Resource Locator) structure to convey context

Routing: mapping URLs to actions

Python Decorators

- Add extra functionality on top of a function
- "@" - decorators before function name
- Effectively, it is a function of a function
 - Take the new inner function as an argument
 - Return a function that does something before calling the inner function

Basic routing in Flask

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return 'Hello, World!'
```

HTTP verbs

```
@app.route('/', methods=['GET'])
def index():
    ...

@app.route('/create', methods=['POST'])
def store():
    ...
```

CRUD-like functionality

```
# Assume 'index', 'store' etc. are functions

@app.route('/', method=['GET'])(index)
@app.route('/create', method=['POST'])(store)
@app.route('/<int:user_id>', method=['GET'])(show)
@app.route('/<int:user_id>/edit', method=['POST'])(update)
@app.route('/<int:user_id>', method=['DELETE'])(destroy)
```

Summary

- Flask is not natively MVC
 - But MVC is more a way of thought than a framework
- Simple URL routing
- Helpers to do large scale routing of common functions

Structure the application with separation of concerns in mind

- MVC is just one way to achieve clean design



Week 6 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 12:36 PM
🔗 Materials	
# Module #	32
▼ Type	Lecture
☰ Week #	6

API Design

Distributed Software Architecture

- Server - Clients
- Standard "protocols" needed for communication
- Assumptions?
 - Server always on?
 - Server knows what client is doing?
 - Client authentication
 - Network latency?

The Web

- Client - Server may be far apart
- Different network, latencies, quality
- Authentication? Not core part of protocol
- State?
 - Server does not know what state client is in
 - Client cannot be sure what state server is in

Architecture for the Web

- Roy Fielding, PhD thesis 2000 UC Irvine

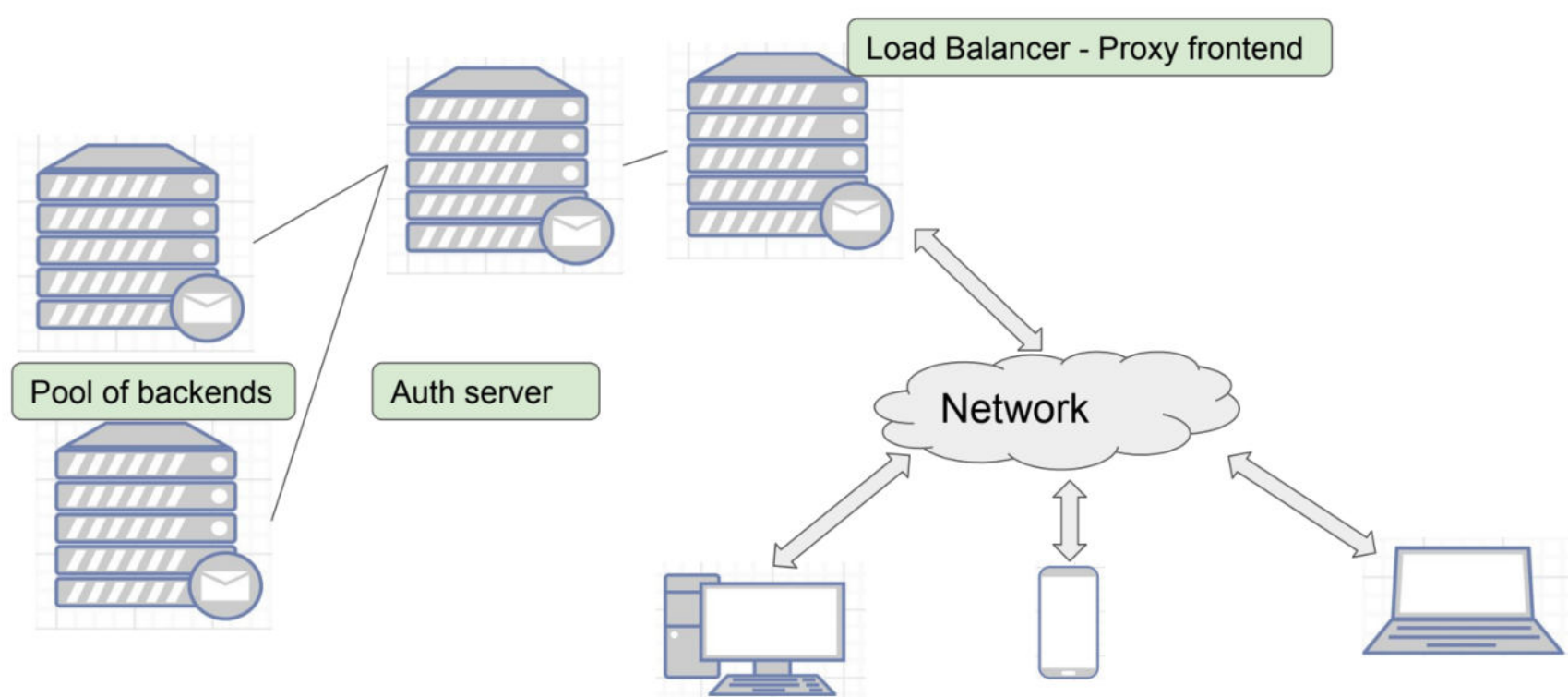
- "REpresentational State Transfer" - REST
 - Take into account the limitations of the web
 - Provide guidelines or constraints
- Software Architecture Style
 - Not a set of rules

Constraint #1: Client-Server

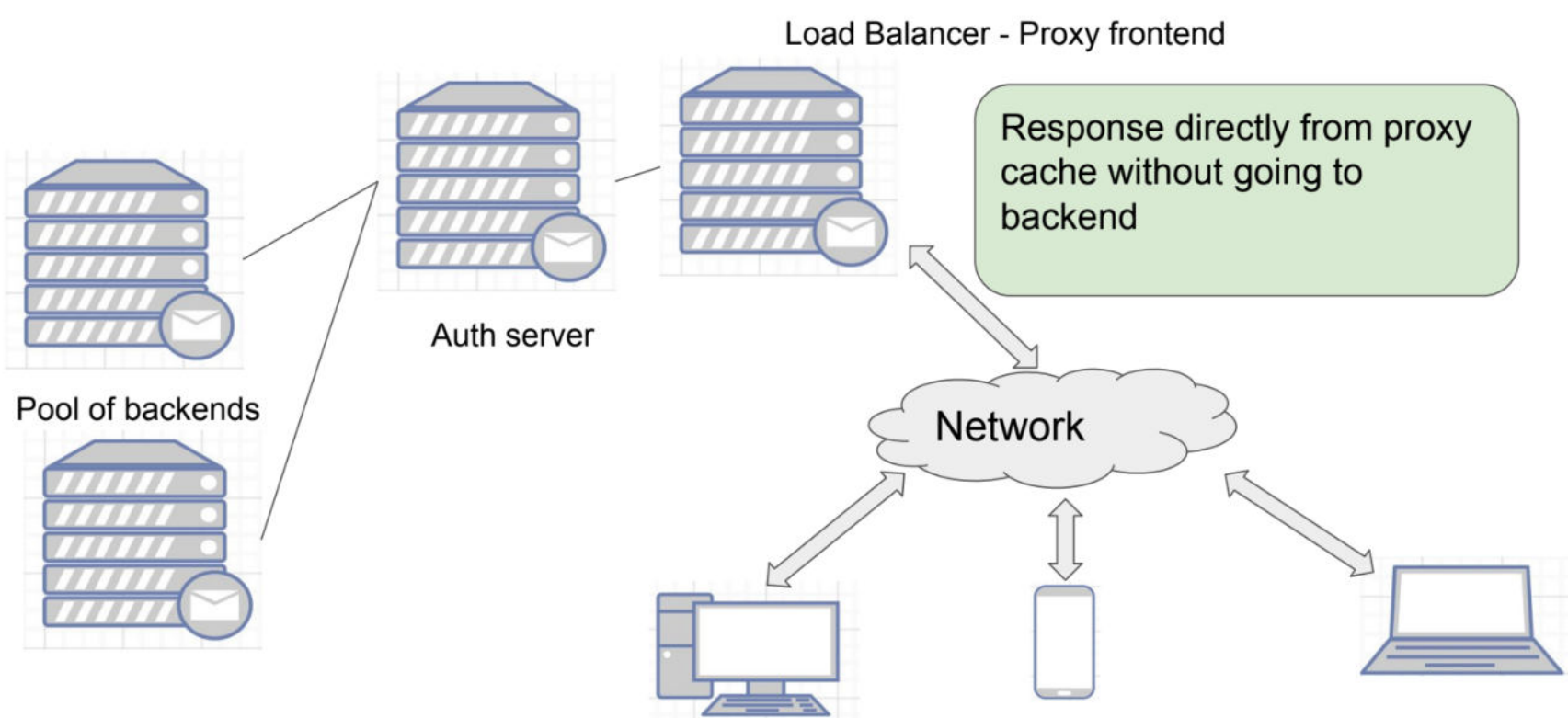
Constraint #2: Stateless

- Server cannot assume state of the client:
 - Which page are you looking at
 - Is a request coming from an already logged in user just because of the address?
- Client cannot assume the state of the server
 - Did the server reboot since the last request?
 - Is this request being answered by the same server?

Constraint #3: Layered system



Constraint #4: Cacheability



Constraint #5: Uniform interface

- Client and State interact in a uniform and predictable manner
- Server exposes "resources"

Hypertext/media used to convey the available resources and functionality - can be discovered by client through hypertext information from the server

Constraint #6: Code on Demand (Optional)

- Server can extend client functionality
 - JavaScript
 - Java applets (are they still relevant?)

Part of the overall structure - these are not hard rules



Week 6 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 7:15 PM
🔗 Materials	
# Module #	33
▼ Type	Lecture
☰ Week #	6

REST

REST

- REpresentational State Transfer

What does that mean?

- State information between client and the server explicitly transferred with every communication

Sequence

- Client accesses Resource identifier from the server
 - Usually URI - superset of URL
 - Typically start from home page of the application
 - No initial state assumed
- Resource Operation specified as part of access
 - If HTTP, then GET,POST, etc.
 - Not fundamentally tied to the protocol
- Server responds with new Resource Identifier
 - New state of system; new links to follow, etc.

State of interaction transferred back and forth

HTTP

- One possible protocol to carry the REST messages

- Use the HTTP verbs to indicate action
- Standardize some types of functionality

- **GET:** Retrieve representation of target resource's state
- **POST:** Enclose data in request: target resource "processes" it
- **PUT:** Create a target resource with data enclosed
- **DELETE:** Delete the target resource

Idempotent Operations

- Repeated application of the operation is not a problem
- **Example:** GET is always safe - read-only operation
- **Example:**
 - **PUT:** will always create the same new resource. If already exists, may give error
 - **DELETE:** can delete only once. May error on repeated deletion, but won't change the data
 - **POST:** May NOT be idempotent
 - **Example:** Add comment to a blog - repeat will cause multiple copies

CRUD

- CRUD: Database operations
- Create, Read, Update, Delete
- Typically a common set of operations needed in most web applications
 - Good candidate of REST based functionality

REST $\neq$ CRUD

But, they do work together quite well

Data Encoding

- **Basic HTML:** for simple reasons
- **XML:** Structured data response
- **JSON:** Simpler form of structured data

Data serialization for transferring complex data types over text based format

JSON

- JavaScript Object Notation
- Nested arrays:
 - Serialize complex data structures like dictionaries, arrays, etc.

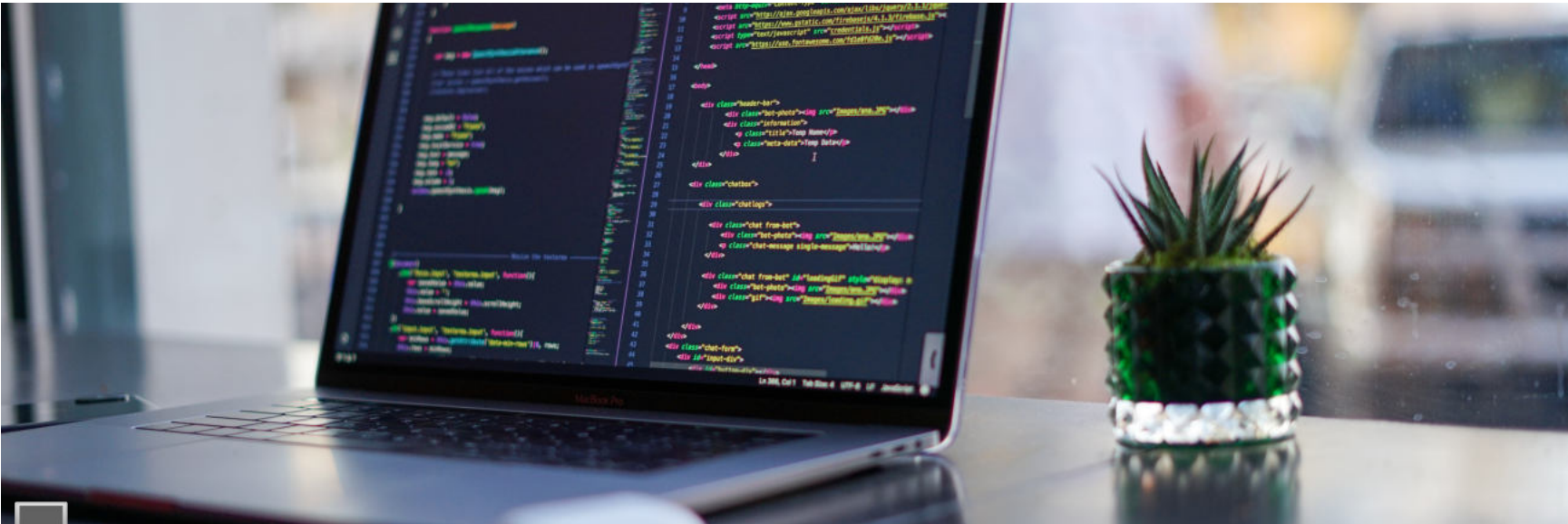
```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```


API data transfer format

- Input to API: text - HTTP
- Output: complex data types - JSON, XML, YAML, etc
 - JSON most commonly used
- Different from internal server representation
- Different from final view presentation

YAML

- **Yet Another Markup Language:** common alternative, especially for documentation and configuration



Week 6 Lecture 3

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 7:17 PM
📎 Materials	
# Module #	34
🔵 Type	Lecture
☰ Week #	6

REST APIs

- CRUD
- Variants of listing
- Specialized functions
 - Create a new virtual machine
 - Reboot an existing virtual machine
 - Turn off street lights on a given street
- Formal specifications help others to use

Example #1: Wikipedia

- Open API
- Search for pages
- History of a page
- JSON output

```
curl "https://en.wikipedia.org/w/rest.php/v1/search/page?q=earth&limit=1"
```

Response

```
{
  "pages": [
    {
      "id": 9228,
      "key": "Earth",
      "title": "Earth",
      "excerpt": "<span class=\"searchmatch\">Earth</span> is the third planet from the Sun and the only astronomical object known to harbour and support life. About 29.2% of <span class=\"searchmatch\">Earth's</span> surface is land, with the remainder being the oceans. The average distance between Earth and the Sun is approximately 149.6 million km (93 million miles). Earth is the second largest planet in the Solar System by both diameter and mass, with a surface area of 510 million km² (197 million sq mi) and a mass of 5.972 × 10²⁴ kg (1.315 × 10²⁵ lb). Earth's gravity keeps the atmosphere in place, and its magnetic field helps shield the planet from the solar wind. Earth's atmosphere and oceans play a crucial role in regulating the planet's temperature and supporting life. The planet is home to a vast number of species, including humans, and is the only known planet to support complex life forms. Earth's history is marked by significant geological events, including the formation of the planet, the development of the atmosphere, and the evolution of life. The planet's future is uncertain, but it is expected to remain habitable for billions of years."
      "description": "Third planet from the Sun in the Solar System",
      "thumbnail": {
        "mimetype": "image/jpeg",
        "size": null,
        "width": 200,
        "height": 200,
        "duration": null,
        "url": "//upload.wikimedia.org/wikipedia/commons/thumb/9/97/The_Earth_seen_from_Apollo_17.jpg/200px-The_Earth_seen_from_Apollo_17.jpg"
      }
    }
  ]
}
```

Documentation

Schema [\[edit \]](#)

id required integer	Page identifier
key required string	Page title in URL-friendly format
title required string	Page title in reading-friendly format
excerpt required string	For <i>search pages endpoint</i>: A few lines giving a sample of page content with search terms highlighted with <code></code> tags For <i>autocomplete page title endpoint</i>: Page title in reading-friendly format
description required string	Short summary of the page topic based on the corresponding entry on Wikidata or <code>null</code> if no entry exists

Detailed Documentation

Search pages [\[edit \]](#)

Route	<code>/search/page?q=search terms</code>	Content type	<code>application/json</code>
Method	<code>GET</code>	Returns	<code>pages</code> object containing array of search results

Searches wiki page titles and contents for the provided search terms, and returns matching pages.

When using this endpoint on your wiki



This endpoint uses the search engine configured in the `$wgSearchType` configuration setting and returns results in the namespaces configured by `$wgNamespacesToBeSearchedDefault`.

Examples [\[edit \]](#)

`curl` `Python` `PHP` `JavaScript`

```
# Search English Wikipedia for up to 20 pages containing information about Jupiter
$ curl https://en.wikipedia.org/w/rest.php/v1/search/page?q=jupiter&limit=20
```

Parameters and Response Codes

Parameters [\[edit \]](#)

q required query	Search terms
limit optional query	Maximum number of search results to return, between 1 and 100. Default: 50

Responses [\[edit \]](#)

200	Success: Results found. Returns a <code>pages</code> object containing an array of search results .
200	Success: No results found. Returns a <code>pages</code> object containing an empty array.
400	Query parameter not set. Add <code>q</code> parameter.
400	Invalid limit requested. Set <code>limit</code> parameter to between 1 and 100.
500	Search error



Week 6 Lecture 4

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 8:07 PM
🔗 Materials	
# Module #	35
▼ Type	Lecture
☰ Week #	6

REST APIs

Example #2: CoWin public APIs

- For Co-Win app: Vaccine registration and information
- Unauthenticated APIs:
 - State-wise search, districts, etc.
- Authenticated APIs:
 - Book appointment

<https://apisetu.gov.in/public/marketplace/api/cowin#/>

General Information

Servers

https://cdn-api.co-vin.in/api - Production Server

Authorize

User Authentication APIs

Metadata APIs

Appointment Availability APIs

Certificate APIs

Schemas

CentersSchema

Example: Availability API

GET

/v2/appointment/sessions/public/findByPin

Get vaccination sessions by PIN

API to get planned vaccination sessions on a specific date in a given pin.

Parameters

Try it out

Name	Description
Accept-Language string (header)	The locate code of the preferred language such as en_US. The text data will be returned in the preferred language along with default English text. Example : hi_IN hi_IN
pincode * required string (query)	110001
date * required string (query)	31-03-2021

Testing public API

WARNING: Do NOT overdo this (This is a public API related to health services, overloading this might have some serious implications especially on someone's life)

```
$ curl -X GET "https://cdn-api.co-vin.in/api/v2/appointment/sessions/public/findByPin"
-H "accept: application/json" -H "Accept-Language: en_US"

{
  "errorCode": "USRRES0001",
  "error": "Input parameter missing"
}
```



```
$ curl -X GET "https://cdn-api.co-vin.in/api/v2/appointment/sessions/public/findByPin?pincode=600020&date=04-08-2021"
-H "accept: application/json" -H "Accept-Language: en_US"
```

```
{"sessions":[
{"center_id":604384,
"name":"Fortis Malar Hospital",
"address":"Fortis Malar HospitalChennai TN.",
"state_name":"Tamil Nadu",
"district_name":"Chennai",
"block_name":"Adyar",
"pincode":600020,
"from":"13:30:00", "to":"15:30:00", "lat":12, "long":80, "fee_type":"Paid",
"session_id":"d40bd2c9-0f42-4948-b794-e3c31fa7c3cc",
"date":"04-08-2021",
"available_capacity":85,
"available_capacity_dose1":40,
"available_capacity_dose2":45,
"fee":"1250", . . .
}
```

```
curl -X GET
"https://cdn-api.co-vin.in/api/v2/registration/certificate/public/download?beneficiary_reference_id=1234567890123" -H "accept: application/json" -H
"Accept-Language: en_US" -H "User-Agent: Mozilla/5.0"
```

Unauthenticated access!

Authentication

- Many APIs must be protected:
 - Only meant for specific users
 - Avoid abuse by overloading servers

How to implement this?

Require a "token" that only a valid user can have

- Securely give token only when the user logs in - Google OAuth, Facebook, etc.
- **API key:** One time token that user downloads - can be copied, so potentially less secure unless combined with other methods

Summary

- **API examples:** CoWin, Google Cloud, Twitter, GitHub,... etc.
- Authentication may be enforced or optional on some parts
- Allows 3rd party integrations
- Equivalent of a "remote procedure call" — call a function on a remote system



Week 6 Lecture 5

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 8:32 PM
🔗 Materials	
# Module #	36
▼ Type	Lecture
☰ Week #	6

OpenAPI

APIs of interest for web apps

- **Purpose:** information hiding — neither server nor the client should know the details of the implementation on the other side
- **Unbreakable contract:** should not change — standardized
 - Versions may update with breaking changes

Documentation

- Highly subjective — some programmers better than others at documenting
- Incomplete — what one programmer finds important may not match others
- Outdated
- Human language specific

Description files

- Machine readable — has very specific structure
- Enable automated processing:
 - Boilerplate code
 - Mock servers
- **Example:** Assembly language is a version of the programming language of computers that is both machine and human readable

- Structured, so that it can be compiled
- **Versus:** English language specification which needs someone to write code

OpenAPI Specification (OAS)

- **Vendor-neutral** format for **HTTP-based remote API** specification
- Does not aim to describe all possible APIs
- Efficiently describe the common use cases
- Originally developed as Swagger — evolved from Swagger 2.0

Current version: OAS3 - v3.1.0 as of Aug. 2021



Week 6 Lecture 6

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 8:49 PM
🔗 Materials	
# Module #	37
▼ Type	Lecture
☰ Week #	6

Important Concepts of an API

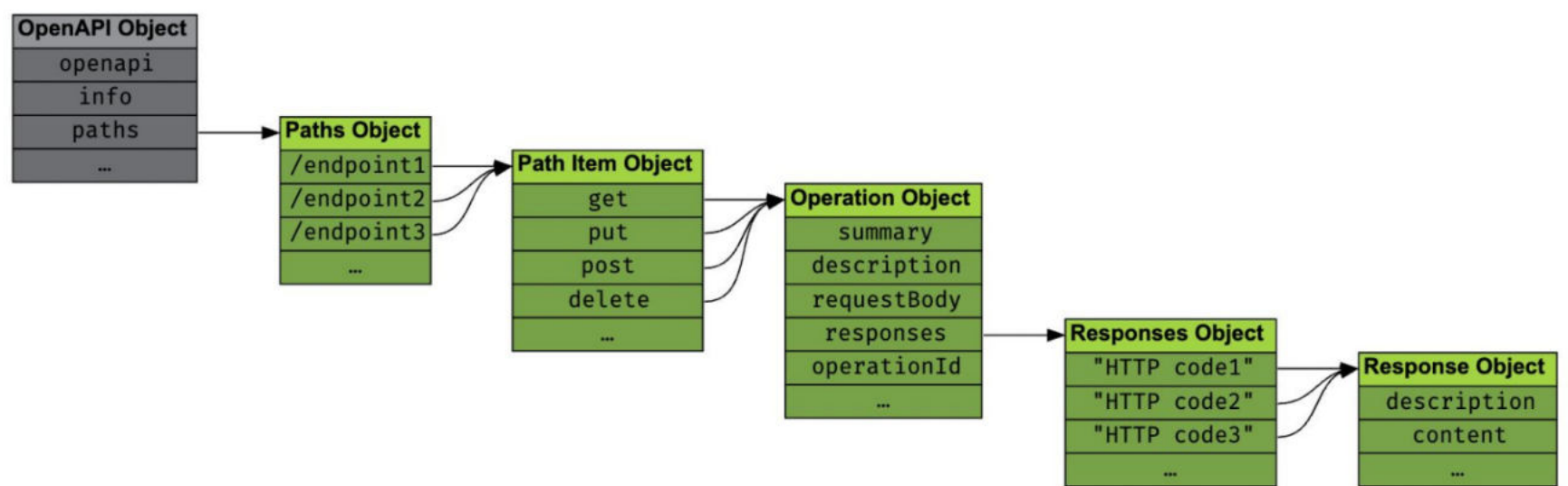
Concepts

- Describe in YAML (or possibly JSON)
- Specific structure to indicate overall information, paths, schemas, etc

eg:

```
openapi: 3.1.0
info:
  title: A minimal OpenAPI document
  version: 0.0.1
paths: {} # No endpoints defined
```

Endpoints List



Source: <https://oai.github.io/Documentation/specification-paths.html>

Paths

```

openapi: 3.1.0
info:
  title: Tic Tac Toe
  description: |
    This API allows writing down marks on a Tic Tac Toe board
    and requesting the state of the board or of individual squares
  version: 1.0.0
paths:
  /board:
  ...

```

Operations

```

paths:
  /board:
    get:
      ...
    put:
      ...

```

Operation object

```

paths:
  /board:
    get:
      summary: Get the whole board
      description: Retrieves the current state of the board and the winner.
      parameters:
        ...
      responses:
        ...

```

Responses

```

paths:
  /board:
    get:
      responses:
        "200":
          ...
        "404":
          ...

```

Response Objects

```

paths:
  /board:
    get:
      responses:
        "200":
          description: Everything went fine.

```



```
content:
  ...
```

Content Specification

```
content:
  application/json:
    ...
  text/html:
    ...
  text/*:
    ...
```

Schema

```
content:
  application/json:
    schema:
      type: integer
      minimum: 1
      maximum: 100
```

Complex schema

```
content:
  application/json:
    schema:
      type: object
      properties:
        productName:
          type: string
        productPrice:
          type: number
```

Parameters

```
paths:
  /users/{id}:
    get:
      parameters:
        - name: id
          in: path
          required: true
```

Request body

```
requestBody:
  content:
    application/json:
      schema:
        type: integer
        minimum: 1
        maximum: 100
```

Best practices

- Design-first v/s Code-first
 - Always prefer the design-first
- Single source of truth
 - The structure of the code should be derived from the OAS - or -
 - Spec should be derived from the code
 - Minimize chances of code and documentation diverging
- Source code version control
- OpenAPI is ... Open — public documentation better to identify the problems

- Automated tools, editors — make use of them